



```
In [ ]: import tensorflow as tf
        from tensorflow.keras import layers
        import numpy as np
        import matplotlib.pyplot as plt
```

```
In [ ]: (train_images, _), (_, _) = tf.keras.datasets.mnist.load_data()

        # Normalize to [-1, 1]
        train_images = (train_images - 127.5) / 127.5

        # Add channel dimension → (28,28,1)
        train_images = train_images.reshape(train_images.shape[0], 28, 28, 1).astype('float32')

        BUFFER_SIZE = 60000
        BATCH_SIZE = 128

        dataset = tf.data.Dataset.from_tensor_slices(train_images)\
            .shuffle(BUFFER_SIZE).batch(BATCH_SIZE)
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
 11490434/11490434 ————— 2s 0us/step

```
In [ ]: def build_generator():
        model = tf.keras.Sequential()

        model.add(layers.Dense(7*7*256, use_bias=False, input_shape=(100,)))
        model.add(layers.BatchNormalization())
        model.add(layers.LeakyReLU())

        model.add(layers.Reshape((7, 7, 256)))

        model.add(layers.Conv2DTranspose(128, (5,5), strides=(1,1), padding='same'))
        model.add(layers.BatchNormalization())
        model.add(layers.LeakyReLU())

        model.add(layers.Conv2DTranspose(64, (5,5), strides=(2,2), padding='same'))
        model.add(layers.BatchNormalization())
        model.add(layers.LeakyReLU())

        model.add(layers.Conv2DTranspose(1, (5,5), strides=(2,2), padding='same',
            use_bias=False, activation='tanh'))

        return model

        generator = build_generator()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
 super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```
In [ ]: def build_discriminator():
```

```

model = tf.keras.Sequential()

model.add(layers.Conv2D(64, (5,5), strides=(2,2), padding='same',
                        input_shape=[28,28,1]))
model.add(layers.LeakyReLU())
model.add(layers.Dropout(0.3))

model.add(layers.Conv2D(128, (5,5), strides=(2,2), padding='same'))
model.add(layers.LeakyReLU())
model.add(layers.Dropout(0.3))

model.add(layers.Flatten())
model.add(layers.Dense(1, activation='sigmoid'))

return model

discriminator = build_discriminator()

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```

super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

```

In [ ]: cross_entropy = tf.keras.losses.BinaryCrossentropy()

def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)

def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    return real_loss + fake_loss

```

```

In [ ]: generator_optimizer = tf.keras.optimizers.Adam(1e-4)
discriminator_optimizer = tf.keras.optimizers.Adam(1e-4)

```

```

In [ ]: noise_dim = 100

def train_step(images):
    noise = tf.random.normal([BATCH_SIZE, noise_dim])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        generated_images = generator(noise, training=True)

        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images, training=True)

        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output, fake_output)

    gradients_of_generator = gen_tape.gradient(gen_loss, generator.trainable_v

```

```
gradients_of_discriminator = disc_tape.gradient(disc_loss, discriminator.t  
  
generator_optimizer.apply_gradients(zip(gradients_of_generator, generator.  
discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator, di
```

```
In [ ]: def generate_and_save_images(model, epoch, test_input):  
    predictions = model(test_input, training=False)  
    fig = plt.figure(figsize=(4,4))  
  
    for i in range(predictions.shape[0]):  
        plt.subplot(4,4,i+1)  
        plt.imshow(predictions[i,::,0] * 127.5 + 127.5, cmap='gray')  
        plt.axis('off')  
  
    plt.suptitle(f'Epoch {epoch}')  
    plt.show()
```

```
In [ ]: EPOCHS = 2  
seed = tf.random.normal([16, noise_dim])  
  
for epoch in range(EPOCHS):  
    for batch in dataset:  
        train_step(batch)  
    show_images(epoch+1, seed)  
    print("Epoch", epoch+1, "Done")
```

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
/tmp/ipython-input-3614842872.py in <cell line: 0>()
      4 for epoch in range(EPOCHS):
      5     for batch in dataset:
----> 6         train_step(batch)
      7     show_images(epoch+1, seed)
      8     print("Epoch", epoch+1, "Done")

/tmp/ipython-input-2850776844.py in train_step(images)
     17     gradients_of_discriminator = disc_tape.gradient(disc_loss, discrimi
nator.trainable_variables)
     18
--> 19     generator_optimizer.apply_gradients(zip(gradients_of_generator, gen
erator.trainable_variables))
     20     discriminator_optimizer.apply_gradients(zip(gradients_of_discrimina
tor, discriminator.trainable_variables))

/usr/local/lib/python3.12/dist-packages/keras/src/optimizers/base_optimizer.py
in apply_gradients(self, grads_and_vars)
    461     def apply_gradients(self, grads_and_vars):
    462         grads, trainable_variables = zip(*grads_and_vars)
--> 463         self.apply(grads, trainable_variables)
    464         # Return iterations for compat with tf.keras.
    465         return self._iterations

/usr/local/lib/python3.12/dist-packages/keras/src/optimizers/base_optimizer.py
in apply(self, grads, trainable_variables)
    525
    526         # Apply gradient updates.
--> 527         self._backend_apply_gradients(grads, trainable_variables)
    528
    529         # Apply variable constraints after applying gradients.
    529         for variable in trainable_variables:

/usr/local/lib/python3.12/dist-packages/keras/src/optimizers/base_optimizer.py
in _backend_apply_gradients(self, grads, trainable_variables)
    591
    592         # Run update step.
--> 593         self._backend_update_step(
    594             grads, trainable_variables, self.learning_rate
    595         )

/usr/local/lib/python3.12/dist-packages/keras/src/backend/tensorflow/optimize
r.py in _backend_update_step(self, grads, trainable_variables, learning_rate)
    118     grads_and_vars = list(zip(grads, trainable_variables))
    119     grads_and_vars = self._all_reduce_sum_gradients(grads_and_vars)
--> 120     tf.__internal__.distribute.interim.maybe_merge_call(
    121         self._distributed_tf_update_step,
    122         self._distribution_strategy,

/usr/local/lib/python3.12/dist-packages/tensorflow/python/distribute/merge_cal
l_interim.py in maybe_merge_call(fn, strategy, *args, **kwargs)
    49     """

```

```

50     if strategy_supports_no_merge_call():
--> 51         return fn(strategy, *args, **kwargs)
52     else:
53         return distribute_lib.get_replica_context().merge_call(

/usr/local/lib/python3.12/dist-packages/keras/src/backend/tensorflow/optimize
r.py in _distributed_tf_update_step(self, distribution, grads_and_vars, learnin
g_rate)
132
133     for grad, var in grads_and_vars:
--> 134         distribution.extended.update(
135             var,
136             apply_grad_to_update_var,

/usr/local/lib/python3.12/dist-packages/tensorflow/python/distribute/distribut
e_lib.py in update(self, var, fn, args, kwargs, group)
3003         fn, autograph_ctx.control_status_ctx(), convert_by_default=False)
3004     with self._container_strategy().scope():
-> 3005         return self._update(var, fn, args, kwargs, group)
3006     else:
3007         return self._replica_ctx_update(

/usr/local/lib/python3.12/dist-packages/tensorflow/python/distribute/distribut
e_lib.py in _update(self, var, fn, args, kwargs, group)
4073     # The implementations of _update() and _update_non_slot() are ident
ical
4074     # except _update() passes `var` as the first argument to `fn()`.
-> 4075     return self._update_non_slot(var, fn, (var,) + tuple(args), kwarg
s, group)
4076
4077     def _update_non_slot(self, colocate_with, fn, args, kwargs, should_grou
oup):

/usr/local/lib/python3.12/dist-packages/tensorflow/python/distribute/distribut
e_lib.py in _update_non_slot(self, colocate_with, fn, args, kwargs, should_grou
p)
4079     # once that value is used for something.
4080     with UpdateContext(colocate_with):
-> 4081         result = fn(*args, **kwargs)
4082         if should_group:
4083             return result

/usr/local/lib/python3.12/dist-packages/tensorflow/python/autograph/impl/api.p
y in wrapper(*args, **kwargs)
594     def wrapper(*args, **kwargs):
595         with ag_ctx.ControlStatusCtx(status=ag_ctx.Status.UNSPECIFIED):
--> 596             return func(*args, **kwargs)
597
598     if inspect.isfunction(func) or inspect.ismethod(func):

/usr/local/lib/python3.12/dist-packages/keras/src/backend/tensorflow/optimize
r.py in apply_grad_to_update_var(var, grad, learning_rate)
129     ):

```

```

130         def apply_grad_to_update_var(var, grad, learning_rate):
--> 131             return self.update_step(grad, var, learning_rate)
132
133         for grad, var in grads_and_vars:

/usr/local/lib/python3.12/dist-packages/keras/src/optimizers/adam.py in update
e_step(self, gradient, variable, learning_rate)
133             variable,
134             ops.divide(
--> 135                 ops.multiply(m, alpha), ops.add(ops.sqrt(v), self.epsilon
on)
136             ),
137         )

/usr/local/lib/python3.12/dist-packages/keras/src/ops/numpy.py in add(x1, x2)
235     if any_symbolic_tensors((x1, x2)):
236         return Add().symbolic_call(x1, x2)
--> 237     return backend.numpy.add(x1, x2)
238
239

/usr/local/lib/python3.12/dist-packages/keras/src/backend/tensorflow/sparse.py
in sparse_wrapper(x1, x2)
491         # x2 is an IndexedSlices, densify.
492         x2 = tf.convert_to_tensor(x2)
--> 493     return func(x1, x2)
494
495     return sparse_wrapper

/usr/local/lib/python3.12/dist-packages/keras/src/backend/tensorflow/numpy.py i
n add(x1, x2)
104     )
105     x1 = convert_to_tensor(x1, dtype)
--> 106     x2 = convert_to_tensor(x2, dtype)
107
108     # Special case of `tf.add`: `tf.nn.bias_add`

/usr/local/lib/python3.12/dist-packages/keras/src/backend/tensorflow/core.py in
convert_to_tensor(x, dtype, sparse, ragged)
144     if dtype is not None:
145         dtype = standardize_dtype(dtype)
--> 146     if not tf.is_tensor(x):
147         if dtype == "bool" or is_int_dtype(dtype):
148             # TensorFlow conversion is stricter than other backends, it
does not

/usr/local/lib/python3.12/dist-packages/tensorflow/python/framework/tensor_util
l.py in is_tf_type(x)
1228     `True` if `x` is a TensorFlow-native type.
1229     """
-> 1230     return isinstance(x, tf_type_classes)
1231
1232

```

```

/usr/lib/python3.12/typing.py in __instancecheck__(cls, instance)
1918         " @runtime_checkable protocols")
1919
-> 1920         if super().__instancecheck__(instance):
1921             return True
1922

```

```

/usr/lib/python3.12/abc.py in __instancecheck__(cls, instance)

```

KeyboardInterrupt:

```

In [ ]: import tensorflow as tf
        from tensorflow.keras import layers
        import numpy as np
        import matplotlib.pyplot as plt

# 1 Load & Preprocess MNIST
(train_images, _), (_, _) = tf.keras.datasets.mnist.load_data()
train_images = (train_images - 127.5) / 127.5
train_images = train_images.reshape(-1, 28, 28, 1).astype('float32')[:20000]

BATCH_SIZE = 128
dataset = tf.data.Dataset.from_tensor_slices(train_images).shuffle(20000).batch(BATCH_SIZE)

# 2 Generator
def build_generator():
    return tf.keras.Sequential([
        layers.Dense(7*7*256, use_bias=False, input_shape=(100,)),
        layers.BatchNormalization(), layers.LeakyReLU(),
        layers.Reshape((7,7,256)),
        layers.Conv2DTranspose(128,5,1,'same',use_bias=False),
        layers.BatchNormalization(), layers.LeakyReLU(),
        layers.Conv2DTranspose(64,5,2,'same',use_bias=False),
        layers.BatchNormalization(), layers.LeakyReLU(),
        layers.Conv2DTranspose(1,5,2,'same',use_bias=False,activation='tanh')
    ])

# 3 Discriminator
def build_discriminator():
    return tf.keras.Sequential([
        layers.Conv2D(64,5,2,'same',input_shape=[28,28,1]),
        layers.LeakyReLU(), layers.Dropout(0.3),
        layers.Conv2D(128,5,2,'same'),
        layers.LeakyReLU(), layers.Dropout(0.3),
        layers.Flatten(), layers.Dense(1, activation='sigmoid')
    ])

generator, discriminator = build_generator(), build_discriminator()

# 4 Loss + Optimizer
cross_entropy = tf.keras.losses.BinaryCrossentropy()
g_loss = lambda fake: cross_entropy(tf.ones_like(fake), fake)
d_loss = lambda real, fake: cross_entropy(tf.ones_like(real), real) + cross_entropy(tf.zeros_like(fake), fake)

```

```

g_opt = tf.keras.optimizers.Adam(1e-4)
d_opt = tf.keras.optimizers.Adam(1e-4)
noise_dim = 100

# 5 Training Step
def train_step(images):
    noise = tf.random.normal([BATCH_SIZE, noise_dim])
    with tf.GradientTape() as gt, tf.GradientTape() as dt:
        fake = generator(noise, training=True)
        real_out = discriminator(images, training=True)
        fake_out = discriminator(fake, training=True)
        gl, dl = g_loss(fake_out), d_loss(real_out, fake_out)
    g_opt.apply_gradients(zip(gt.gradient(gl, generator.trainable_variables),
                             d_opt.apply_gradients(zip(dt.gradient(dl, discriminator.trainable_variables),

# 6 Visualization
def show_images(epoch, seed):
    preds = generator(seed, training=False)
    fig = plt.figure(figsize=(4,4))
    for i in range(16):
        plt.subplot(4,4,i+1)
        plt.imshow(preds[i,:,:,0]*127.5+127.5, cmap='gray')
        plt.axis('off')
    plt.suptitle(f'Epoch {epoch}')
    plt.show()

# 7 Training Loop
EPOCHS = 10
seed = tf.random.normal([16, noise_dim])

for epoch in range(EPOCHS):
    for batch in dataset:
        train_step(batch)
    show_images(epoch+1, seed)
    print("Epoch", epoch+1, "Done")

```

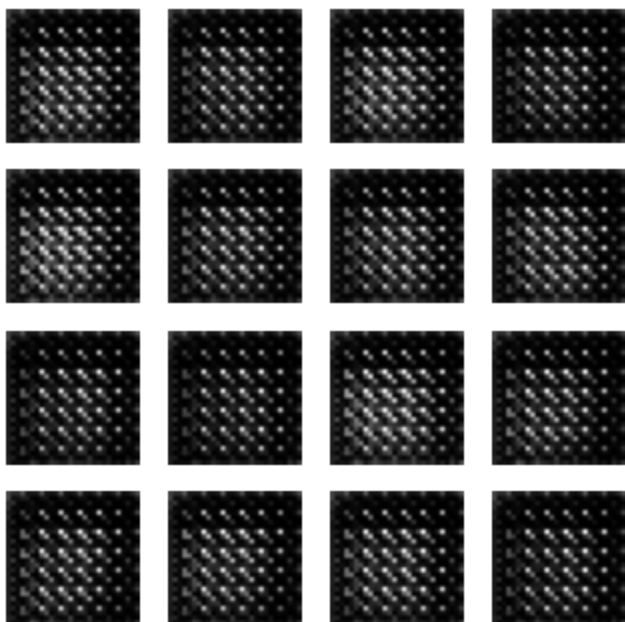
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

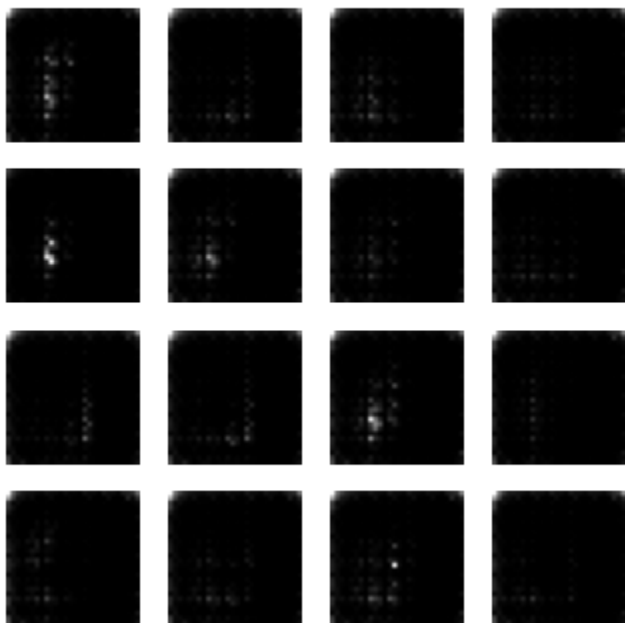
```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```


Epoch 1



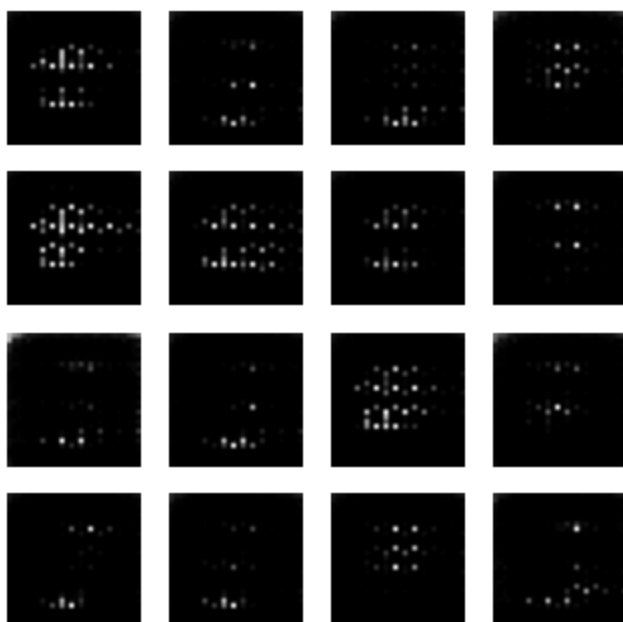
Epoch 1 Done

Epoch 2



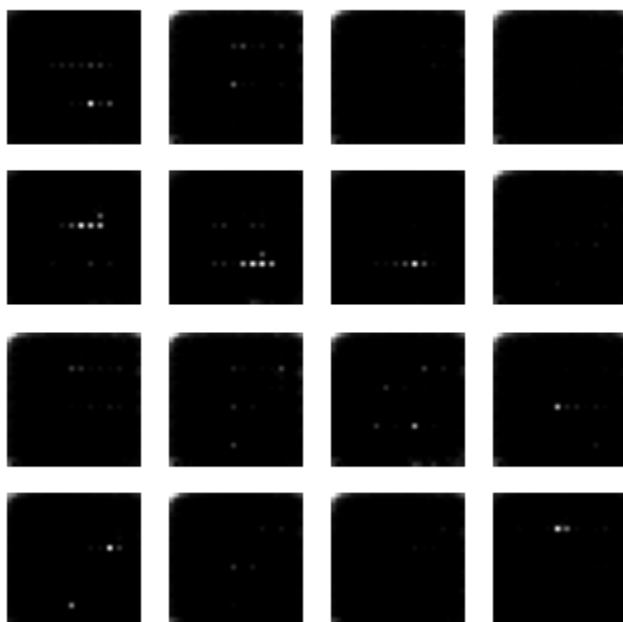
Epoch 2 Done

Epoch 3



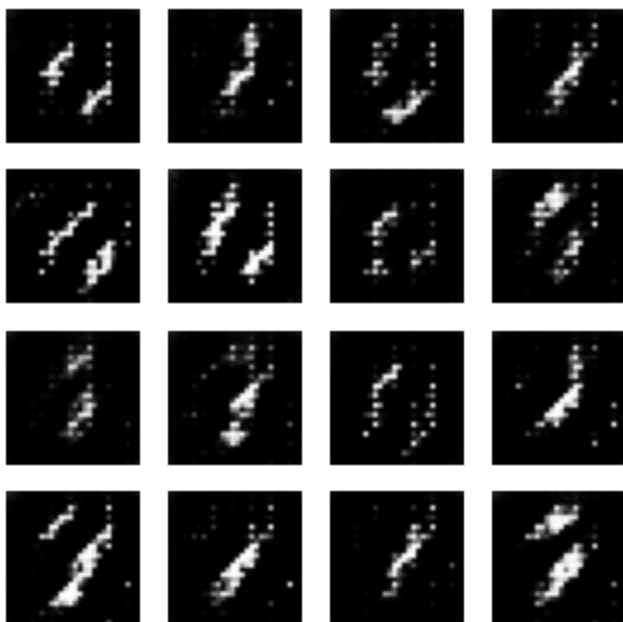
Epoch 3 Done

Epoch 4



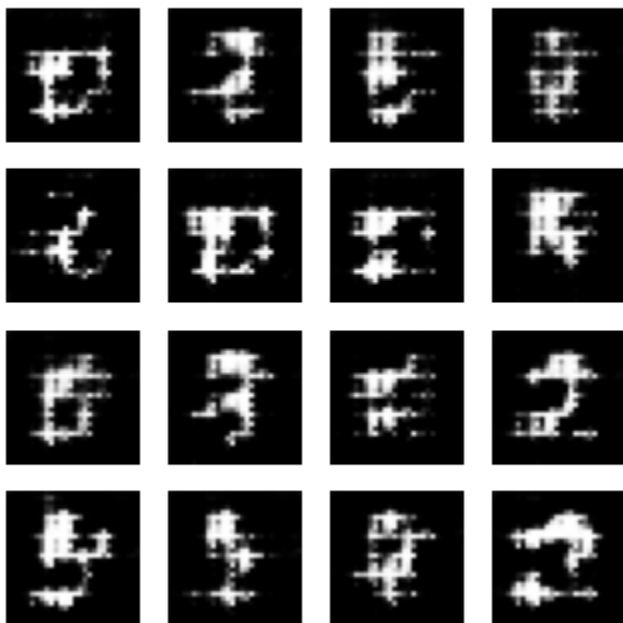
Epoch 4 Done

Epoch 5



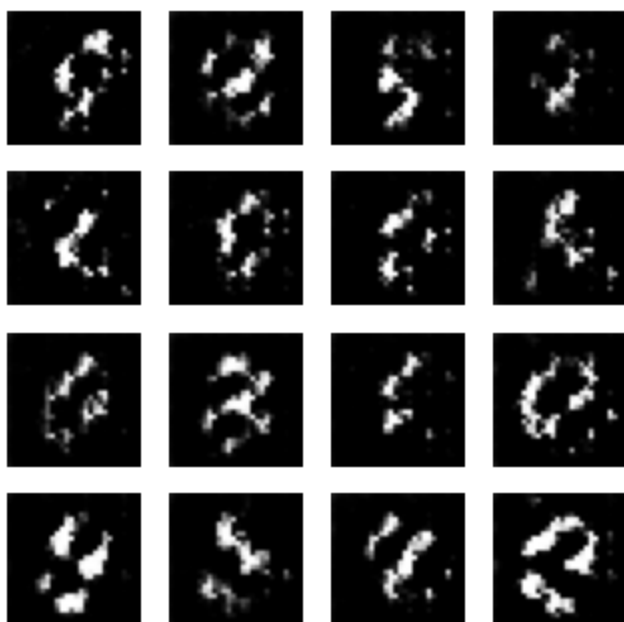
Epoch 5 Done

Epoch 6



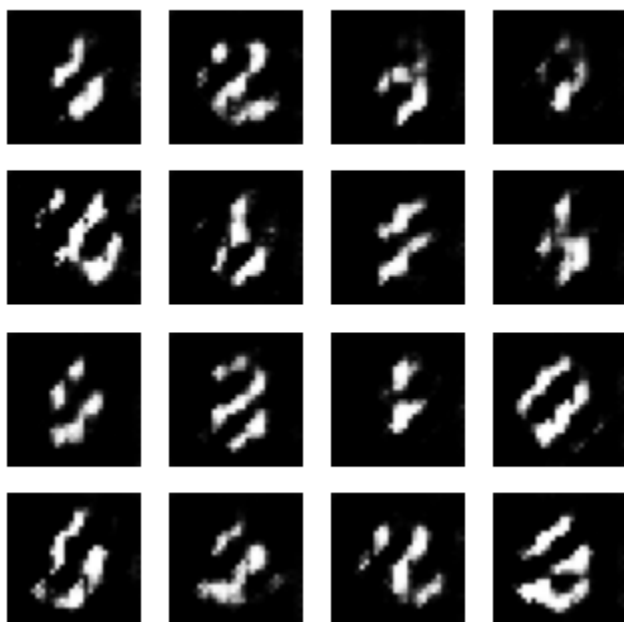
Epoch 6 Done

Epoch 7



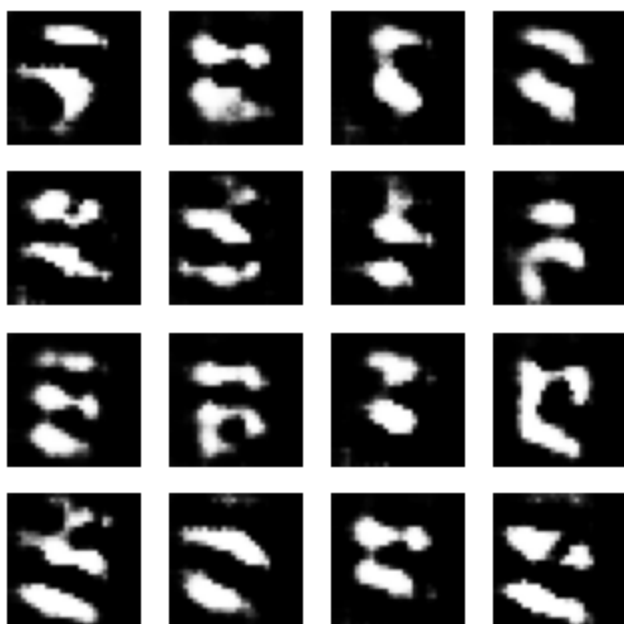
Epoch 7 Done

Epoch 8



Epoch 8 Done

Epoch 9



Epoch 9 Done

Epoch 10



Epoch 10 Done